

Design and Implementation of a Database Management System for Crop Planning and Farm Input Management System — Review 2

DBMS Project-Based Learning — Review 2: Normalized Design, DDL & Queries

Sufiyan Yusuf Jahan · Roll No. 25WU0102276

Worsen University — School of Technology · B.Tech CSE (AI & ML)

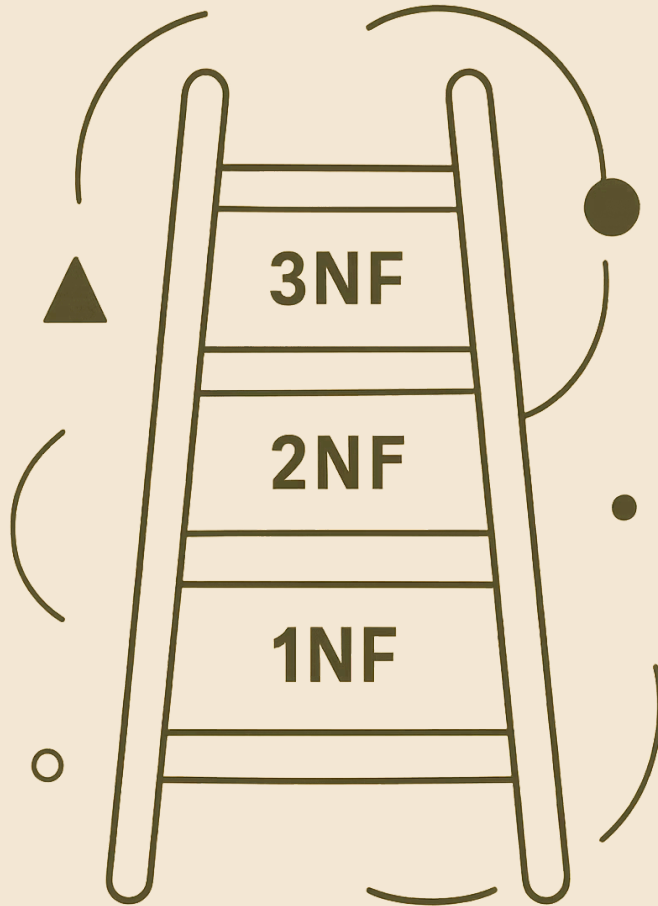


Recap: Where We Left Off (Review 1)

Problem statement: support precision crop planning and tracked farm input management for small-to-medium farms. Review 1 produced: 13 core entities, an initial ER diagram, and a draft schema. Review 2 converts that design into a constraint-enforced relational database ready for queries and demos.



Normalization to 3NF – Goal & Result



01

1NF → Atomic rows

Eliminate repeating groups; each field stores one value (e.g., single variety per catalog row).

02

2NF → Remove partial dependencies

Composite-key dependencies moved to dedicated tables so attributes depend on whole PK.

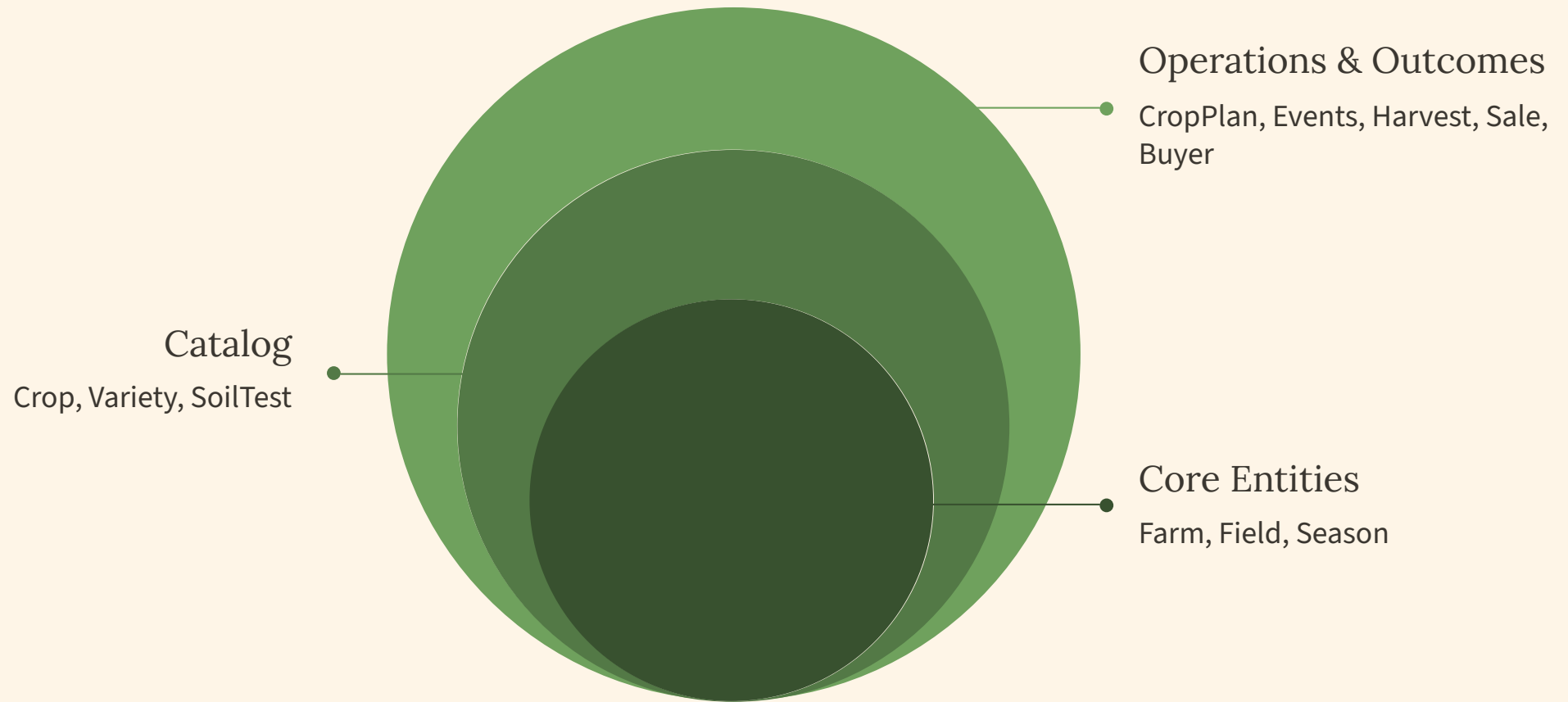
03

3NF → Remove transitive dependencies

Facts stored exactly once: crop & variety details live in Catalog tables (Crop, Variety), not duplicated in CropPlan.

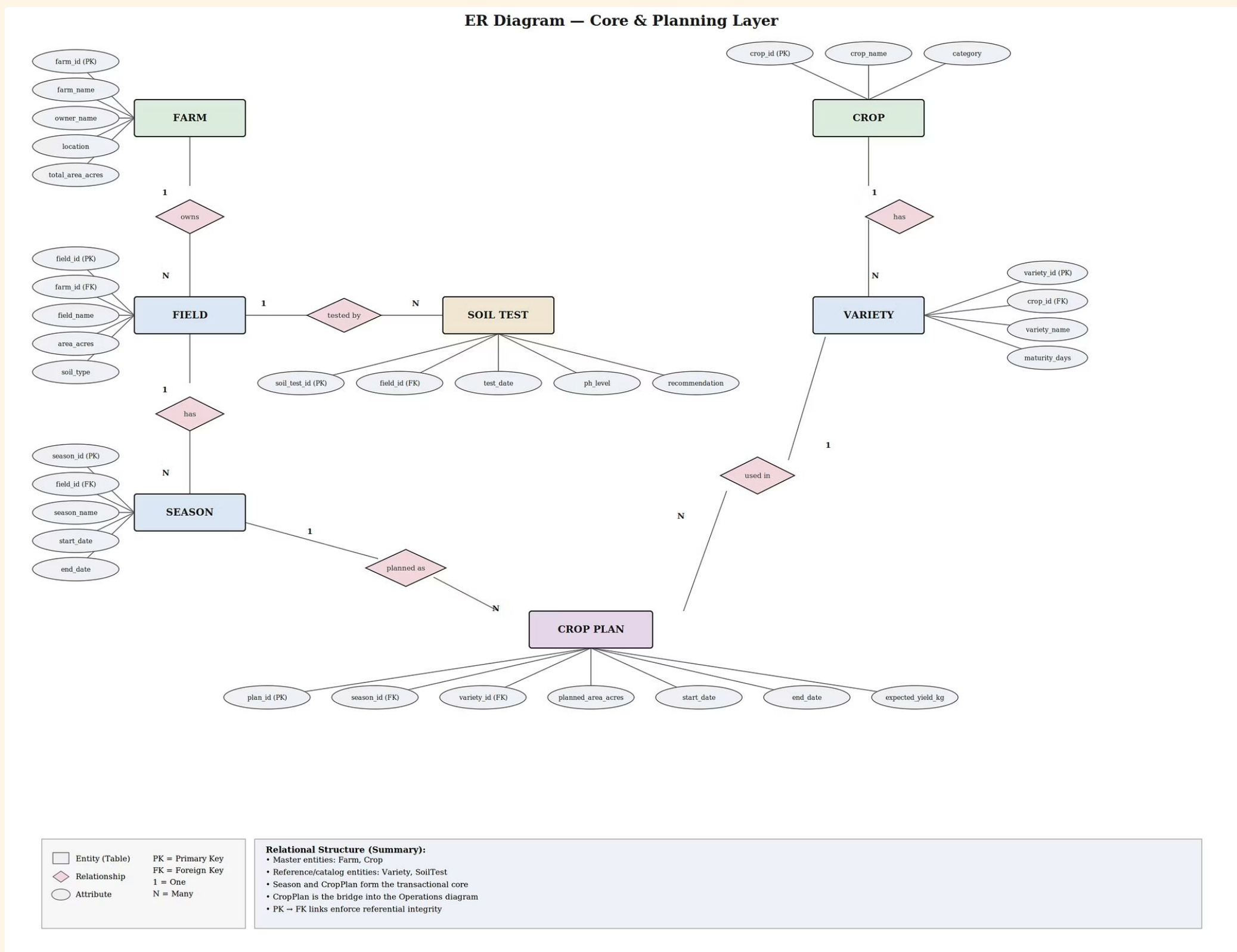
Outcome: concise tables, predictable joins, and simplified constraint logic for correctness and updates.

Final ER Diagram



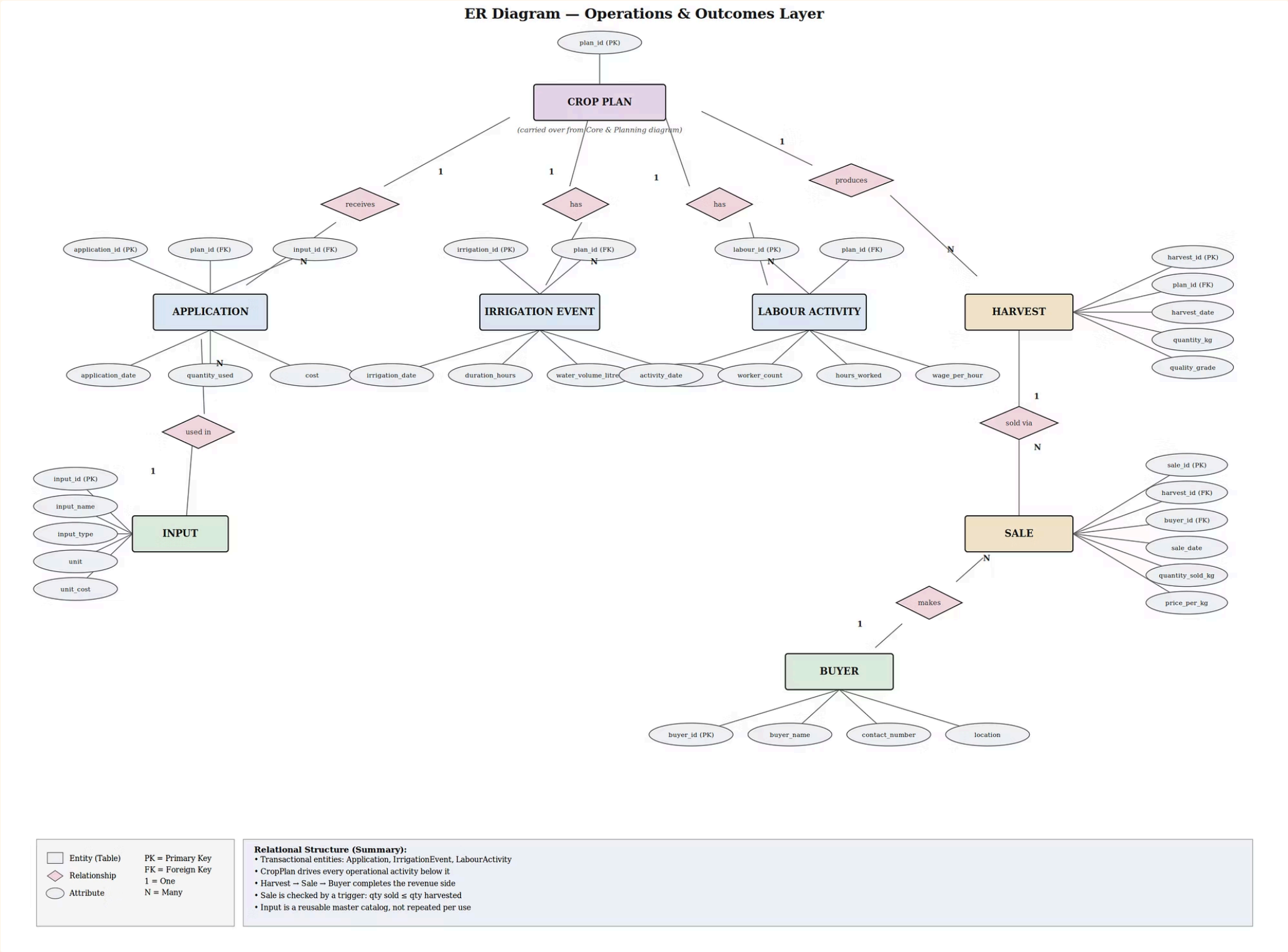
Flows reflect lifecycle from farm setup to market sale; relationships enforce referential integrity and support audit trails for inputs and events.

ER Diagram – Core & Planning Layer



This diagram shows the foundational entities (Farm, Field, Season) and the planning layer (Crop, Variety, SoilTest, CropPlan) with their relationships and attributes. The color-coded legend indicates entity types, relationships, and cardinality constraints.

ER Diagram – Operations & Outcomes Layer



This diagram illustrates the operational and outcome entities (CropPlan, Application, IrrigationEvent, LabourActivity, Harvest, Input, Sale, Buyer) and their interconnections. It captures the complete lifecycle from input application through harvest to market sale, with all foreign key relationships clearly marked.

Relational Schema – 14 Tables (grouped)

Layer	Table	Keys (PK; FK)
Core	Farm	farm_id (PK)
	Field	field_id (PK); farm_id (FK)
	Season	season_id (PK); farm_id (FK)
Catalog	Crop	crop_id (PK)
	Variety	variety_id (PK); crop_id (FK)
	SoilTest	soittest_id (PK); field_id (FK)
Planning	CropPlan	plan_id (PK); season_id (FK); variety_id (FK)
Operations & Outcomes	Input	input_id (PK)
	Application	application_id (PK); plan_id (FK); input_id (FK)
	IrrigationEvent	irrigation_id (PK); plan_id (FK)
	LabourActivity	labour_id (PK); plan_id (FK)
	Harvest	harvest_id (PK); plan_id (FK)
	Buyer	buyer_id (PK)
	Sale	sale_id (PK); harvest_id (FK); buyer_id (FK)

Each table includes sensible indexes and FK constraints to maintain referential integrity and optimize joins.

Enforcing Business Rules: Constraints

Row-level rules — CHECK constraints

- `area_hectares > 0`
- `quantity >= 0; price >= 0`
- `soil_ph BETWEEN 0 AND 14`
- `season.start_date < season.end_date`

Implemented directly in DDL where rules reference a single row.

Cross-table rules — TRIGGERS

- CropPlan start/end must fall within its Season dates (INSERT/UPDATE trigger)
- Application and IrrigationEvent timestamps must lie within CropPlan active window
- Total sold quantity from Harvest cannot exceed harvested_quantity (before INSERT on Sale)

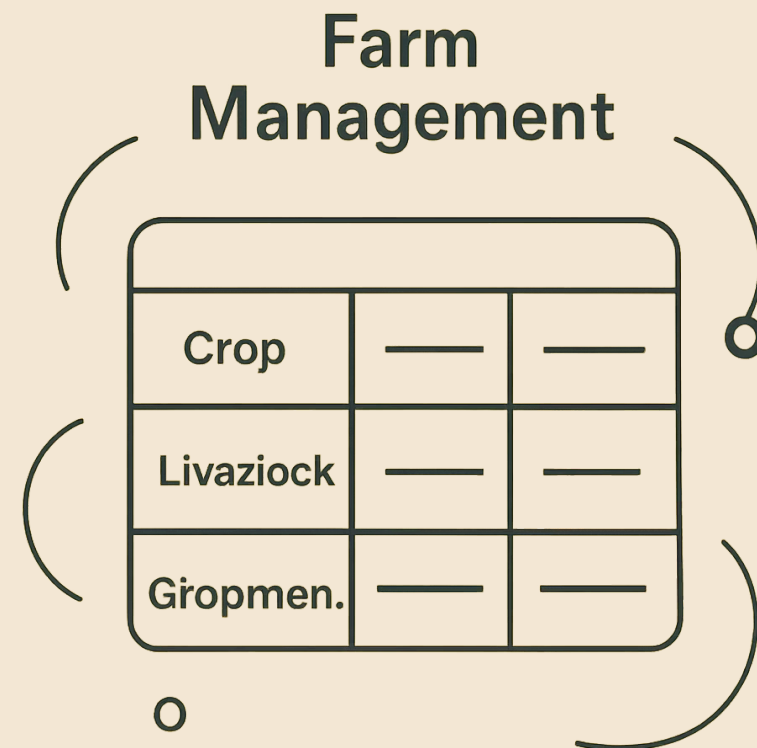
Why triggers: CHECK constraints are single-row only — multi-row or cross-table invariants require procedural enforcement.

Sample Data Overview

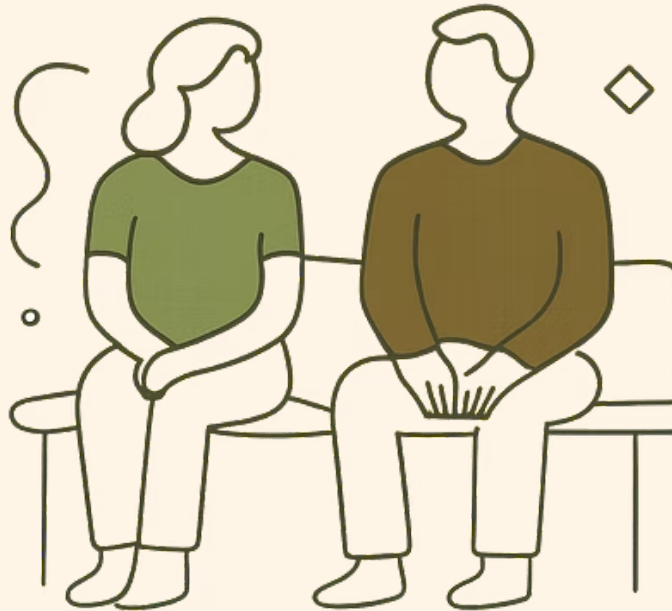
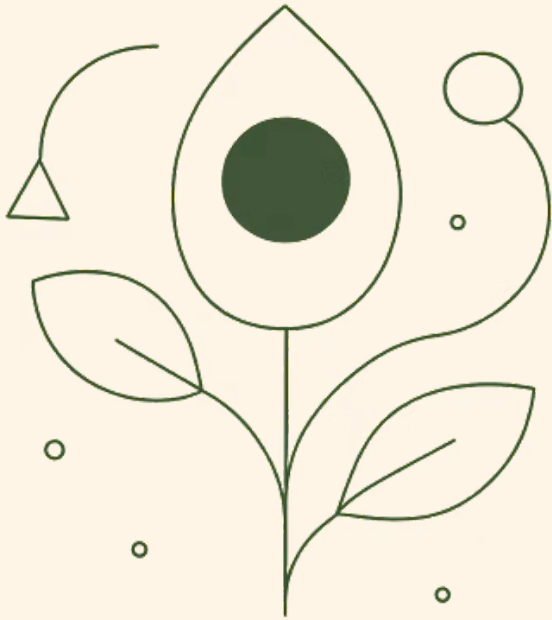
Populated dataset summary (demonstration-ready):

- 2 Farms, 3 Fields (perimeter + soil characteristics)
- 3 Seasons spanning planting windows
- 3 CropPlans: Maize (high-yield), Cotton (fiber), Groundnut (legume)
- Inputs: fertilizers, seed batches, pesticides with unit costs
- Operational events: irrigation logs, labour entries, input applications
- Outcomes: Harvest records with expected vs actual yield; Sales linked to Buyers

This dataset is sufficient to exercise joins, aggregates, triggers, subqueries, and the CropProfitability view.



Filled Tables — Live Demo Screenshots (placeholders)



Frames represent sanitized outputs from MySQL client: neat column headings, limited rows for readability. These support the live demo of queries and trigger behavior.

Query Execution — Joins & Aggregates

Query - Full field plan (join)

```
mysql> SELECT f.farm_name, fl.field_name, s.season_name, c.crop_name, v.variety_name, cp.start_date, cp.end_date, cp.expected_yield_kg
-> FROM CropPlan cp
-> JOIN Season s ON cp.season_id = s.season_id
-> JOIN Field fl ON s.field_id = fl.field_id
-> JOIN Farm f ON fl.farm_id = f.farm_id
-> JOIN Variety v ON cp.variety_id = v.variety_id
-> JOIN Crop c ON v.crop_id = c.crop_id;
```

Result:

farm_name	field_name	season_name	crop_name	variety_name	start_date	end_date	expected_yield_kg
Green Valley Farm	North Block	Kharif 2026	Maize	DHM-117	2026-06-10	2026-09-30	6000.00
Green Valley Farm	South Block	Kharif 2026	Cotton	Bt Cotton RCH-2	2026-06-15	2026-10-25	4500.00
Sunrise Agro	Riverside Plot	Rabi 2025-26	Groundnut	TAG-24	2025-11-10	2026-03-01	3000.00

3 rows in set (0.001 sec)

Answers: Which farm, field, season, crop and variety each plan belongs to — plus its dates and expected yield.

Costs per CropPlan (aggregate)

```
mysql> SELECT cp.plan_id, c.crop_name, SUM(a.cost) AS total_input_cost
-> FROM CropPlan cp
-> JOIN Application a ON cp.plan_id = a.plan_id
-> JOIN Variety v ON cp.variety_id = v.variety_id
-> JOIN Crop c ON v.crop_id = c.crop_id
-> GROUP BY cp.plan_id, c.crop_name;
```

Result:

plan_id	crop_name	total_input_cost
1	Maize	9070.00
2	Cotton	4800.00
3	Groundnut	2000.00

3 rows in set (0.004 sec)

Answers: "Total input and labour cost per crop plan (for profitability).

Query Execution — Nested Query & View

Nested query — yield exceptions

```
mysql> SELECT cp.plan_id, c.crop_name, cp.expected_yield_kg, h.quantity_kg AS actual_yield_kg
-> FROM CropPlan cp
-> JOIN Variety v ON cp.variety_id = v.variety_id
-> JOIN Crop c ON v.crop_id = c.crop_id
-> JOIN Harvest h ON h.plan_id = cp.plan_id
-> WHERE h.quantity_kg > (SELECT cp2.expected_yield_kg FROM CropPlan cp2 WHERE cp2.plan_id = cp.plan_id);
```

Result:

```
+-----+-----+-----+-----+
| plan_id | crop_name | expected_yield_kg | actual_yield_kg |
+-----+-----+-----+-----+
|      1 | Maize     |          6000.00 |          6200.00 |
+-----+-----+-----+-----+
1 row in set (0.003 sec)
```

Finds plans where actual harvest exceeded expectation — useful for model validation.

CropProfitability VIEW

```
mysql> CREATE OR REPLACE VIEW CropProfitability AS
-> SELECT
->   cp.plan_id,
->   c.crop_name,
->   v.variety_name,
->   COALESCE(input_cost.total, 0) AS total_input_cost,
->   COALESCE(labour_cost.total, 0) AS total_labour_cost,
->   COALESCE(sale_rev.total, 0) AS total_sale_revenue,
->   COALESCE(sale_rev.total, 0)
->     - COALESCE(input_cost.total, 0)
->     - COALESCE(labour_cost.total, 0) AS net_profit
-> FROM CropPlan cp
-> JOIN Variety v ON cp.variety_id = v.variety_id
-> JOIN Crop c ON v.crop_id = c.crop_id
-> LEFT JOIN (
->   SELECT plan_id, SUM(cost) AS total
->   FROM Application GROUP BY plan_id
-> ) AS input_cost ON input_cost.plan_id = cp.plan_id
-> LEFT JOIN (
->   SELECT plan_id, SUM(worker_count * hours_worked * wage_per_hour) AS total
->   FROM LabourActivity GROUP BY plan_id
-> ) AS labour_cost ON labour_cost.plan_id = cp.plan_id
-> LEFT JOIN (
->   SELECT h.plan_id, SUM(sl.quantity_sold_kg * sl.price_per_kg) AS total
->   FROM Harvest h JOIN Sale sl ON sl.harvest_id = h.harvest_id
->   GROUP BY h.plan_id
-> ) AS sale_rev ON sale_rev.plan_id = cp.plan_id;
Query OK, 0 rows affected (0.016 sec)
```

Result:

```
mysql> SELECT * FROM CropProfitability;
+-----+-----+-----+-----+-----+-----+-----+
| plan_id | crop_name | variety_name | total_input_cost | total_labour_cost | total_sale_revenue | net_profit |
+-----+-----+-----+-----+-----+-----+-----+
|      1 | Maize     | DHM-117      |          9070.00 |          2900.000 |         111000.0000 |  99030.0000 |
|      2 | Cotton    | Bt Cotton RCH-2 |          4800.00 |          2730.000 |         279500.0000 | 271970.0000 |
|      3 | Groundnut | TAG-24       |          2000.00 |          1320.000 |         154000.0000 | 150680.0000 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.003 sec)
```

Provides an at-a-glance profitability metric answering: "Is this crop making money?"